Epsilon: A Bring-Your-Own-Data Research Platform with Trusted Verification

Nizomjon Khajiev
nizomjon.khajiev@monash.edu
Action Lab, Monash University
Melbourne, Australia

Aare Puussaar
aare.puussaar@monash.edu
Action Lab, Monash University
Melbourne, Australia

Lee Shen Chu
leeshen.chu@monash.edu
Action Lab, Monash University
Malaysia
Malaysia

Patrick Olivier
patrick.olivier@monash.edu
Action Lab, Monash University
Melbourne, Australia

Lay-Ki Soon
soon.layki@monash.edu
Action Lab, Monash University
Malaysia
Malaysia

Delvin Varghese
delvin.varghese@monash.edu
Action Lab, Monash University
Melbourne, Australia

Abstract

We present Epsilon, an open-source platform for analyzing sensitive research data with per-execution verification. Epsilon addresses three practical barriers faced by researchers using Trusted Research Environments (TREs): (1) researchers experience challenges in using their own datasets without uploading data to platform infrastructure; (2) manual policy review creates bottlenecks in the submission workflow; and (3) existing TEE platforms provide only platform-level attestation with no per-execution proof binding specific code to specific datasets. Epsilon adopts a zero-trust model: no component outside the enclave is trusted, enabling secure cross-institutional data analysis without requiring institutions to share raw data.

Epsilon introduces *Archetypes*, a bring-your-own-data mechanism by which researchers link their datasets without persistent data transfer. Only database credentials are stored, in an encrypted vault; data is not persisted on platform storage and is processed ephemerally within the TEE. An automated policy pre-filtering layer screens submissions before human review. A *Verification Center* extends standard TEE platform attestation with per-execution dataset and code binding, where the enclave passes a hash bundle (code, data, output, timestamp) to the Nitro Secure Module, which signs it as part of the hardware attestation document, producing portable verification receipts.

Source code, SDK, and research templates are available on GitHub.[1]

[1] https://github.com/Epsilon-Data

This work is licensed under a Creative Commons Attribution 4.0 International License.
SysTEX '26, Edinburgh, Scotland Uk
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2607-1/2026/04
https://doi.org/10.1145/3805690.3805733

CCS Concepts: • **Security and privacy** → *Domain-specific security and privacy architectures*; **Embedded systems security**; **Systems security**.

Keywords: Trusted Execution Environments, Verification Center, Research Data Infrastructure, Confidential Computing, Zero-trust

ACM Reference Format:
Nizomjon Khajiev, Aare Puussaar, Lee Shen Chu, Patrick Olivier, Lay-Ki Soon, and Delvin Varghese. 2026. Epsilon: A Bring-Your-Own-Data Research Platform with Trusted Verification. In *9th Workshop on System Software for Trusted Execution (SysTEX '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3805690.3805733

1 Introduction

Trusted Research Environments (TREs) enable analysis of sensitive data by bringing computation to data rather than data to researchers [1]. Despite widespread adoption, researchers face persistent practical barriers [2, 3]:

- **No Bring-Your-Own-Data.** Researchers must use platform-curated datasets. Analyzing institutional data requires uploading it to platform infrastructure, creating legal and governance barriers [2]. A study across 29 institutions found that almost three-quarters of researchers reported difficulties accessing or managing data for their own projects [12].
- **Manual Policy Bottleneck.** Each code submission requires human review against dataset policies. UK Biobank's approval process averaged 24 weeks from application to data release as of 2018 [11]; this does not scale with research demands [4]. Furthermore, platforms like OpenSAFELY prohibit entire method categories, such as neural networks, random forests, and LLMs because output checkers cannot evaluate disclosure risk in models with large parameter spaces [13]. TEE-based sandboxing removes this restriction.

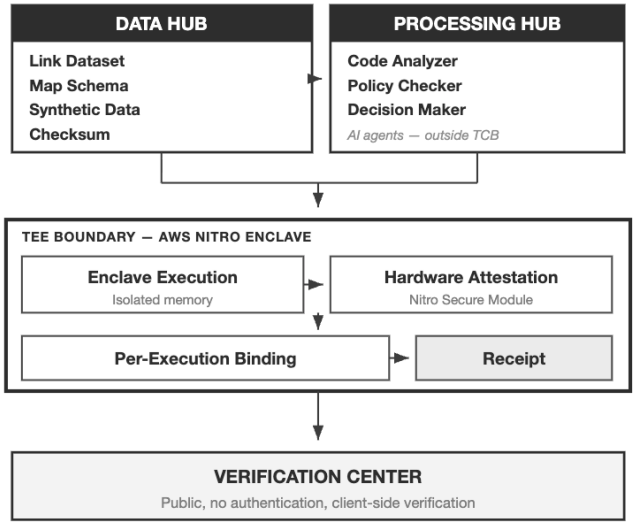

Figure 1. Epsilon platform architecture. Researchers link datasets via Archetypes (Data Hub), receive automated policy feedback (Processing Hub), and execute code in AWS Nitro Enclaves with TEE-signed verification receipts (Security Layer).

- **Platform-Only Attestation.** TEE platforms verify that an application runs in a secure enclave but provide no per-execution proof binding specific code to specific datasets [5]. As a result, researchers cannot independently prove their analysis ran as intended.

We present Epsilon, an open-source platform that addresses all three barriers with three contributions: (1) *Archetypes*—reusable semantic data structures inspired by archetype-based modelling [14]—for BYOD dataset linking without persistent data transfer; (2) per-execution binding via Nitro attestation, extending platform-level TEE guarantees to individual research jobs; and (3) a public Verification Center enabling third-party verification without platform trust or authentication.

Tool Availability. A live instance is at https://app.epsilon-data.org; SDK and source code are at https://github.com/Epsilon-Data.

2 Tool Architecture

Epsilon comprises three components (Figure 1): a Data Hub for dataset linking and schema management, a Processing Hub for automated code screening, and a Security Layer for verified execution.

2.1 BYOD via Archetypes

Existing TREs require data to be uploaded to platform infrastructure for permanent storage. Epsilon inverts this: *data is not persisted on Epsilon storage*. Epsilon separates three roles: *data owners* publish datasets and control access; *researchers* request access and submit code; the *platform* mediates execution but never sees plaintext data. At execution time,

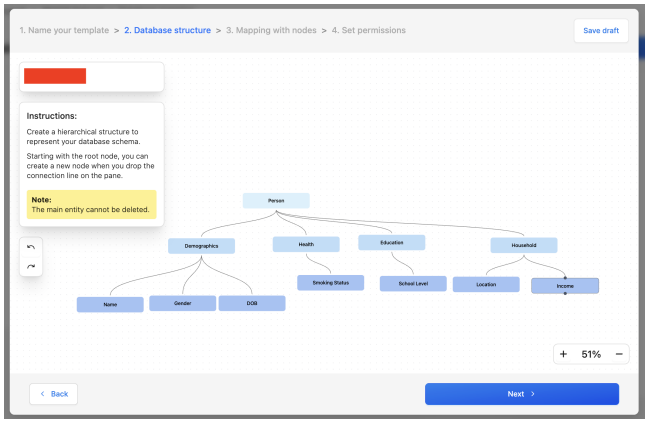

Figure 2. Archetype builder interface showing a health surveillance dataset mapped to a semantic hierarchy: `Person` → `Demographics` (Name, Gender, DOB), `Health` (Smoking Status), `Education` (School Level), `Household` (Location, Income). Data is not persisted on Epsilon storage.

a source-side encryption proxy (`epsilon-proxy`), a Go binary on the data owner's machine, verifies the enclave's attestation, queries the local database, encrypts results with the enclave's attested public key, and tunnels ciphertext to the enclave (see Section 3.4). Data enters only the enclave's isolated memory and is discarded after execution.

Researchers define data structures through a visual node-leaf mapping interface (Figure 2):

1. **Link Dataset.** Researchers provide database credentials (stored encrypted in Vault).
2. **Map Columns.** Researchers map database columns to semantic entities using a node-leaf hierarchy (e.g., `Patient` → `Encounter` → `Observation`).
3. **Generate Synthetic Data.** The system infers types and generates synthetic data matching the schema for local development.

Each Archetype produces a versioned checksum verified by the TEE at execution time. Schema metadata changes are recorded in Apache Atlas for lineage tracking.

2.2 Automated Policy Agents

TREs require manual human review for every code submission—a process that does not scale with research demand [4]. Epsilon automates pre-screening using a multi-agent pipeline built on CrewAI [6]: a *Code Analyzer* checks for dangerous patterns (network access, file I/O, system calls), a *Policy Checker* validates compliance against dataset-specific rules, and a *Decision Maker* produces a final recommendation (Approve, Conditional, or Reject) with explanations. Review intensity is tiered by data sensitivity: Tier 1 (synthetic) requires minimal checks, Tier 2 (aggregated) permits AI-only approval, and Tier 3 (individual-level) mandates AI analysis plus human review.

Researchers submit analysis code via Git: they fork a project template, develop locally against synthetic data, and

push to their repository. Epsilon fetches the submitted code by commit SHA, ensuring the exact version is recorded in the attestation and publicly auditable.

Agents operate at two stages: *pre-execution* (code structure, PII patterns, policy compliance) and *post-execution* (statistical disclosure control, output risk scoring). While systems such as OpenSAFELY require dual independent manual output checking by trained reviewers for every result release [1], Epsilon's post-execution agents automate the initial screening, reserving human review for flagged outputs and Tier 3 data. The current prototype uses GPT-4 via API, meaning submitted code is transmitted to an external service during pre-screening—a limitation incompatible with TRE confidentiality requirements. This serves as a proof-of-concept for the multi-agent architecture; production deployments will migrate to a locally-hosted open-source LLM (candidate models: Qwen2.5-Coder-7B, DeepSeek-R1-Distill-32B), and for sensitive deployments the AI pre-screening layer can be disabled entirely without affecting TEE attestation guarantees, since all agents operate explicitly *outside the Trusted Computing Base*, reducing manual review workload without bypassing verification. Following zero-trust principles, every execution receives the same per-execution attestation regardless of an agent's recommendation.

2.3 Verification Center

Standard TEE platforms provide platform-level attestation: hardware identity, runtime image integrity, and key management. These guarantees prove that *an application* runs in genuine TEE, but not that a *specific dataset* was processed by *specific code* [5]. Epsilon's Verification Center adds four per-execution layers on top of standard platform attestation (Table 1). Critically, all cryptographic verification runs *client-side* in the visitor's browser using WebCrypto API—a compromised server cannot produce false results (Appendix F).

Table 1. Verification Center layers. Standard layers are provided by any TEE deployment. Epsilon adds four per-execution research provenance layers.

	Verification Layer	Origin
1	TEE Hardware Attestation	Standard
2	Analysis Code Hash	**Extended**
3	Runtime Image (PCR0)	Standard
4	Network Encryption	Standard
5	Key Management (Vault)	Standard
6	Dataset Integrity Hash	**Epsilon**
7	AI Policy Compliance‡	**Epsilon**
8	Result Binding (output ⟵ code + data)	**Epsilon**
9	Immutable Ledger Record	**Planned**

‡Informational only; outside the Trusted Computing Base.

Layer 2 (Extended). Standard platforms verify a container image hash (PCR0). Epsilon additionally includes the specific analysis script hash (`main.py`) in the `user_data` field, providing artifact-level granularity beyond image-level measurement.

Layers 6–9 (Epsilon additions). *Dataset Integrity*: SHA-256 hash of the input data snapshot. A baseline hash is generated when the dataset is first connected; at execution time, the enclave recomputes the hash and compares it against the baseline to detect any modification. No existing platform fingerprints the dataset. *AI Policy*: automated compliance result from policy agents, recorded as metadata (outside TCB; informational, not a security guarantee). *Result Binding*: at execution completion, the enclave constructs a JSON hash bundle (code hash, data hash, output hash, job ID, nonce, timestamp) and passes it as the `user_data` field in an NSM `GetAttestationDocument` call. The Nitro Secure Module signs the entire attestation document—including `user_data`—using a certificate chain rooted in the AWS Nitro Attestation PKI (ECDSA P-384). Because `user_data` is signed by hardware the host OS cannot access, no component outside the enclave can forge or modify the hash bundle after signing. This binds specific code to specific data to specific output in a single TEE-signed attestation. *Ledger*: the verification receipt is designed for submission to a public ledger for immutable storage; the current prototype stores receipts in a platform database (ledger integration is in progress).

3 Implementation and Usage

3.1 Technology Stack

Table 2. Implementation technology stack.

Component	Technology
Frontend	React, TypeScript
Backend	Python, Express.js, AWS Lambda
Policy Agents	CrewAI, GPT-4†
TEE	AWS Nitro Enclaves
Key Exchange	RSA-2048 OAEP (per-job keypair)
Secrets	HashiCorp Vault

†Migrating to locally-hosted LLM; see Section 2.2.

TEE Selection. We chose AWS Nitro Enclaves [9] because (1) they offer a production-grade alternative to Intel SGX/TDX on a different trust model; and (2) they integrate natively with Epsilon's existing AWS infrastructure (EC2, KMS, IAM).

On-Demand Enclave Provisioning. Epsilon provisions enclave-enabled EC2 instances on demand via EventBridge + Lambda and auto-terminates after 10 minutes idle. Table 3 reports cold-start versus warm latency.

3.2 Threat Model

We assume three adversary classes: (1) an *honest-but-curious cloud provider* that can observe network traffic and storage but cannot penetrate TEE hardware; (2) a *malicious researcher* attempting data exfiltration or policy evasion through submitted code; and (3) *compromised platform infrastructure*

Table 3. Performance measurements (m5.xlarge instance). Mean of N=6 jobs across two datasets: synthetic (13 rows) and demographic health surveillance (10,647 rows, 6 tables). N=5 cold boots. Larger-scale evaluation is planned; current measurements characterize baseline operational latency.

Operation	Cold	Warm
EC2 + enclave boot	31 s	—
Data ingestion (Lambda → vsock)	7.4 s	7.4 s
Script execution (in enclave)	53 ms	53 ms
Attestation generation (NSM)	9 ms	9 ms
Server-side verification	125 ms	125 ms
Client-side verification	~1 ms	~1 ms
End-to-end	**~49 s**	**~18 s**

Attestation document: 4.8 KB ; verification receipt: 821 B.

Boot time range: 28–33 s. Attestation + verification: <150 ms (constant).

where any component outside the enclave may be controlled by an attacker.

We explicitly trust AWS as the hardware provider. The TCB consists solely of the Nitro Enclave image; all other components are untrusted. Any image modification changes PCR0 and fails attestation. Researcher code runs sandboxed with no network access. Data enters encrypted with the enclave's ephemeral key. Side-channel attacks are out of scope.

3.3 SDK Usage

Epsilon provides a CLI-first Python SDK: `epsilon init` generates typed classes from the Archetype schema with synthetic data for local testing; `epsilon build` packages and submits the analysis for enclave execution. Upon completion, the researcher receives results with a TEE-signed verification receipt. The complete workflow is shown in Appendix G.

3.4 Trust Boundaries

Source-Side Encryption. Since the original submission, we developed `epsilon-proxy`, a Go binary that data owners run locally. The proxy verifies the enclave's attestation (COSE_Sign1, certificate chain, PCR0) before sending data, encrypts query results with the enclave's attested public key (RSA-2048-OAEP + AES-256-CBC), and tunnels ciphertext to the enclave. No platform component sees plaintext data, eliminating the TCB boundary limitation noted in Section 2. We chose ephemeral credential-based access over encrypt-and-upload to avoid persistent ciphertext on platform storage.

Additional Boundaries. The researcher's database must be reachable at execution time—no cached copies exist. Dataset linking records an audit trail in the verification receipt. The attestation proves hash binding, not computation correctness.

4 Demonstration

We demonstrate the platform using a demographic health surveillance dataset (10,647 persons across 5,149 households, 6 relational tables) from an active research collaboration. The dataset contains sensitive fields including date of birth, national ID numbers, GPS coordinates, health conditions, and smoking status—data that cannot be transferred outside the host institution.

Walkthrough.

1. **Link & Map.** The researcher provides PostgreSQL credentials (stored in Vault; no data transferred) and maps columns via the Archetype interface: `Person` → `Demographics`, `Health`, `Education`, `Household`.
2. **Develop.** Epsilon generates synthetic data matching the schema. The researcher develops analysis scripts locally (e.g., smoking prevalence by gender, education level distribution).
3. **Submit & Execute.** Code is submitted for automated policy pre-filtering, then executes inside a Nitro Enclave. The ingestion service fetches real data using stored credentials, encrypts it with the enclave's public key, and delivers it via vsock. The enclave processes all 10,647 records in isolated memory and produces a signed attestation binding code, data, and output hashes.
4. **Verify.** The researcher receives results plus a TEE-signed verification receipt viewable on the Verification Center dashboard. Journal editors or ethics boards can independently verify the execution via receipt ID.

Appendix B shows the complete Archetype workflow (column mapping, permissions, published dataset) and a live job execution with three-stage progress tracking (Figure 6).

Adoption. The entire workflow from `pip install` to verified execution requires no TEE expertise from the researcher. The SDK abstracts enclave provisioning, key exchange, and attestation verification; researchers interact only with Python classes generated from their Archetype. The Verification Center requires no authentication, enabling third-party auditing without platform accounts.

5 Related Tools

Table 4. Comparison with existing platforms. ○ = planned.

Platform	BYOD	Auto	TEE	Per-Exec	Ledger
OpenSAFELY [1]	×	×	×	×	×
UK Biobank [7]	×	×	×	×	×
Leonhard Med [8]	×	×	×	×	×
ManaTEE [10]	×	×	✓	×	×
Phala/dstack [5]	×	×	✓	×	✓
Epsilon	✓	✓	✓	✓	○

Non-TEE platforms represent the TREs researchers currently use; the TEE and Per-Exec columns highlight the gap

Epsilon fills. ManaTEE [10] targets single-organization sharing with no BYOD or per-execution verification. Phala's dstack [5] provides TEE verification but at *boot-time and platform-level* only [5]. PraaS [17] provides enclave-signed proofs of property using SGX; Epsilon applies a similar pattern to the TRE domain with dataset binding and public verification. The mechanism is TEE-agnostic.

6 Conclusion

Epsilon extends TEE platform attestation with per-execution verification, enabling researchers to link datasets without persistent transfer and obtain portable receipts binding specific code to specific data. Future work: direct enclave-to-database connections, locally-hosted policy LLM, immutable ledger integration, and TEE portability to Intel TDX and Arm CCA.

References

[1] Williamson, E.J., et al. 2020. Factors associated with COVID-19-related death using OpenSAFELY. *Nature* 584, 7821 (2020), 430–436.
[2] Kavianpour, S., Sutherland, J., Mansouri-Benssassi, E., Coull, N., and Jefferson, E. 2022. Next-Generation Capabilities in Trusted Research Environments: Interview Study. *Journal of Medical Internet Research* 24, 9 (2022), e33720.
[3] Graham, M., Milne, R., Fitzsimmons, P., and Sheehan, M. 2023. Trust and the Goldacre Review: why trusted research environments are not about trust. *Journal of Medical Ethics* 49, 10 (2023), 670–673.
[4] Rod, A.B., Zhou, B., and Rousseau, M-E. 2023. There's no "I" in Research Data Management: Reshaping RDM Services Toward a Collaborative Multi-Stakeholder Model. *Journal of eScience Librarianship* 12, 1 (2023), e624.
[5] Zhou, S., Wang, K., and Yin, H. 2025. Dstack: A Zero Trust Framework for Confidential Containers. *arXiv preprint* arXiv:2509.11555.
[6] Moura, J. 2024. CrewAI: Framework for orchestrating role-playing AI agents. github.com/joaomdmoura/crewAI
[7] Sudlow, C., et al. 2015. UK Biobank: An open access resource. *PLOS Medicine* 12, 3 (2015), e1001779.
[8] Okoniewski, M.J., et al. 2024. Leonhard Med, a trusted research environment for processing sensitive research data. *Journal of Integrative Bioinformatics* 21, 3 (2024), 20240021.
[9] Amazon Web Services. 2023. AWS Nitro Enclaves User Guide.
[10] Jaiswal, V., Lee, D., and Sun, M. 2024. PrivacyGo Data Clean Room: A new tool for data collaboration. *TikTok for Developers Blog*. Donated as ManaTEE to the Confidential Computing Consortium, Linux Foundation, October 2024. github.com/manatee-project/manatee
[11] Conroy, M., et al. 2019. The advantages of UK Biobank's open-access strategy for health research. *Journal of Internal Medicine* 286, 4 (2019), 389–397.
[12] McCracken, C. and MacDougall, R. 2025. Researcher Challenges and Experiences with Data Services. *Ithaka S+R*. DOI: 10.18665/sr.322388.
[13] OpenSAFELY. 2024. Policies for Researchers: Analytical Methods. https://www.opensafely.org/policies-for-researchers/. Accessed February 2026.
[14] Beale, T. 2002. Archetypes: Constraint-based Domain Models for Future-proof Information Systems. In *OOPSLA Workshop on Behavioral Semantics*.
[15] gvisor-tap-vsock: Networking for VMs and containers using a TAP interface over vsock. https://github.com/containers/gvisor-tap-vsock
[16] Brave Software. nitriding-daemon: A toolkit for running applications in AWS Nitro Enclaves with networking and attestation support. https://github.com/brave/nitriding-daemon
[17] Bailleu, M., Giantsidi, D., Gavrielatos, V., Katebzadeh, M.A., and Pramod, B. 2024. PraaS: Verifiable Proofs of Property as-a-Service with Intel SGX. In *Proceedings of the 7th Workshop on System Software for Trusted Execution (SysTEX '24)*.

A Open Science Statement

Live Platform. A working instance is accessible at https://app.epsilon-data.org. The Verification Center is at https://trust.epsilon-data.org. The demonstration allows users to create Archetypes, submit code for policy checking, and observe verification receipt generation.

SDK. The Epsilon SDK is publicly available:

- `pip install epsilon-sdk` (v0.3.4)
- Attestation verifier: `npm install @epsilon-data/nitro-verify`
- Research template: https://github.com/Epsilon-Data/epsilon-research-template

Reproducibility. The enclave image build scripts, attestation verification tools, source-side encryption proxy (`epsilon-proxy`), and deployment configurations are available as open-source artifacts at https://github.com/Epsilon-Data.

B Archetype Workflow and Job Execution

Figures 3–6 show the complete workflow from column mapping through job execution.

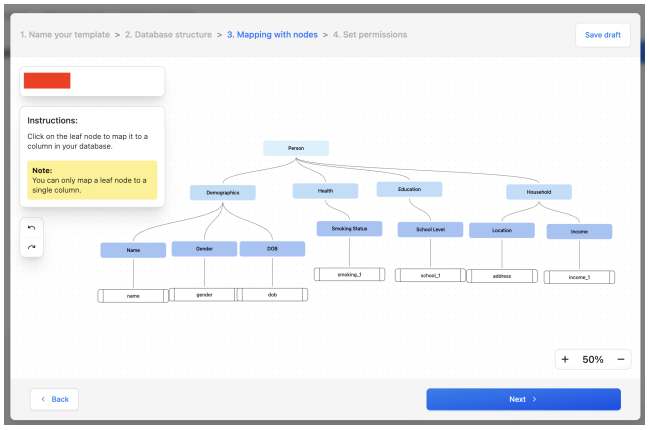

Figure 3. Column mapping step. Leaf nodes are mapped to database columns (e.g., `Gender` → `gender`, `Smoking Status` → `smoking_1`). The system uses these mappings to generate typed Python classes and synthetic data.

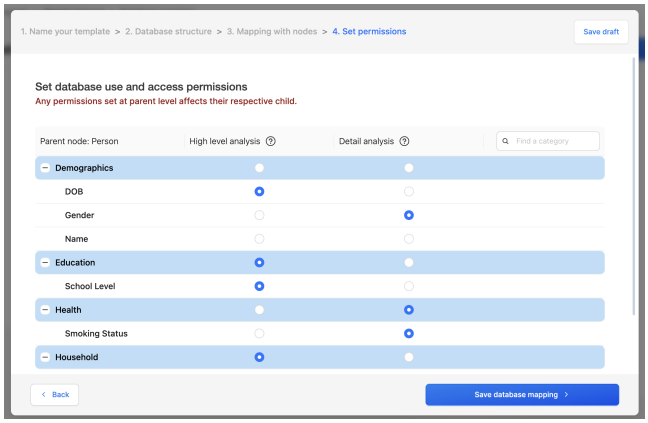

Figure 4. Access permission configuration. Data owners set per-field sensitivity levels: *High-level analysis* (aggregated statistics only) or *Detailed analysis* (individual-level access). Permissions propagate from parent nodes to children.

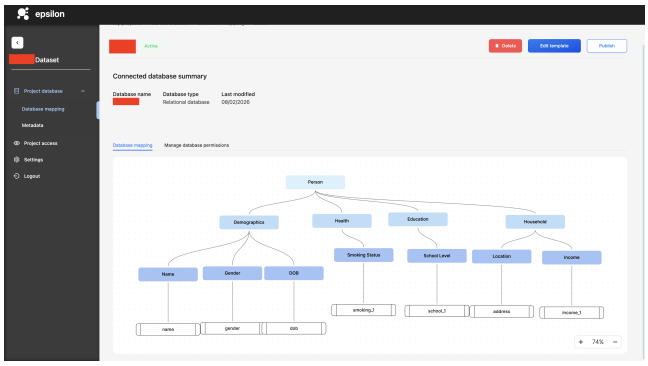

Figure 5. Published dataset overview showing the connected database summary, Archetype tree visualization, and management controls. The dataset is marked "Active" and available for researcher access requests.

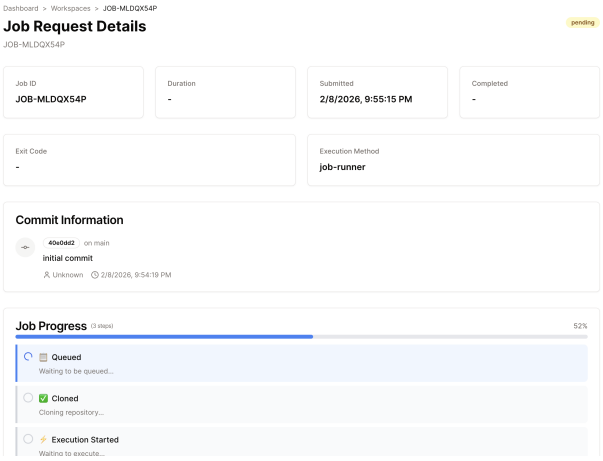

Figure 6. Job execution detail page showing commit information, three-stage progress tracking (Queued → Cloned → Execution Started), and execution metadata. Upon completion, TEE-signed attestation and verification receipt are attached.

C AI Policy Analysis

Figure 7 shows the output of the multi-agent policy pre-screening pipeline (Section 2.2). The CrewAI agents produce a structured decision with confidence score, reasoning explanation, identified risks, and recommendations. The AI agents are explicitly outside the TCB—their output is informational and does not affect attestation guarantees.

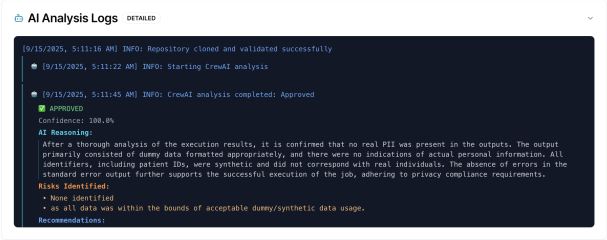

Figure 7. AI policy analysis output showing structured decision (Approve/Conditional/Reject), confidence score, reasoning, and risk assessment. All agents operate outside the Trusted Computing Base.

D Trust Boundary Architecture

Figure 8 illustrates the trust boundary between untrusted platform components and the TEE-protected execution environment. The Client, Data Hub, and AI Agents reside in the untrusted zone. All data entering the TEE boundary is encrypted with the enclave's ephemeral public key. The Execution and Attestation modules operate within the TEE, where data is decrypted and processed in isolated memory inaccessible to the host operating system.

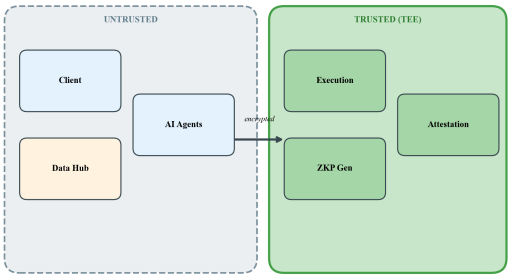

Figure 8. Trust boundary architecture. Grey (dashed): untrusted platform components. Green (solid): TEE-protected modules. Data crosses the boundary only in encrypted form.

E Verification Receipt Schema

Each job execution produces two artifacts: (1) a raw COSE_Sign1 attestation document generated and signed inside the enclave by the Nitro Secure Module (the TEE-trusted artifact), and (2) a structured verification receipt generated server-side by parsing the attestation (a convenience wrapper, not part of the TCB). The receipt below shows the parsed result:

```
{
  "verified_at": "2026-02-08T12:00:00Z",
```

```
  "valid": true,
  "checks": {
    "syntax_valid": true,
    "certificate_chain_valid": true,
    "signature_valid": true,
    "pcr_verified": true,
    "output_verified": true
  },
  "pcrs": {
    "pcr0": "be7f3bfa8c460b...",
    "pcr1": "4b4d5b3661b3efc...",
    "pcr2": "c810e8d6dbf7800..."
  },
  "user_data": {
    "job_id": "JOB-MDWQIM1G",
    "script_hash": "2783a6934dd993e...",
    "dataset_hash": "f14f363065506...",
    "output_hash": "a1b2c3d4e5f6...",
    "timestamp": "2026-02-08T12:00:00Z"
  },
  "module_id": "i-076...aef-enc019...",
  "verifier_version": "1.0.0"
}
```

Listing 1. Verification receipt JSON schema. The `checks` object records five independent verification steps.

Dual verification model. The platform implements two independent verification paths: (1) *server-side* verification at execution time using a Python verifier (`cbor2`, `cryptography`), producing the receipt above; and (2) *client-side* verification in the researcher's browser using a TypeScript verifier (`cborg`, WebCrypto API). Both verifiers implement identical five-step validation: CBOR/COSE parsing, certificate chain verification against the AWS Nitro root CA, ECDSA P-384 signature verification, PCR comparison, and output hash matching. Client-side verification uses `allowExpired=true` for historical attestations whose short-lived certificates (valid ∼3 hours) have expired since execution time.

F Verification Center

The Verification Center is a public-facing web application where anyone researchers, reviewers, ethics boards can independently verify that a job executed inside a genuine AWS Nitro Enclave. No login is required. The design is inspired by Phala Network's Trust Center [5] but adapted for AWS Nitro attestation with per-execution verification.

F.1 Public Verification Ledger

Figure 9 shows the homepage, which serves as a public ledger of all verified executions. The dashboard displays aggregate statistics (total verified, verified today, success rate) and three verification pillars: Hardware Attestation, Enclave Image Verification, and Output Integrity. Below, a paginated table lists all completed jobs with their verification status, PCR match status, commit SHA, and timestamp. Any visitor can click a job to inspect its full attestation.

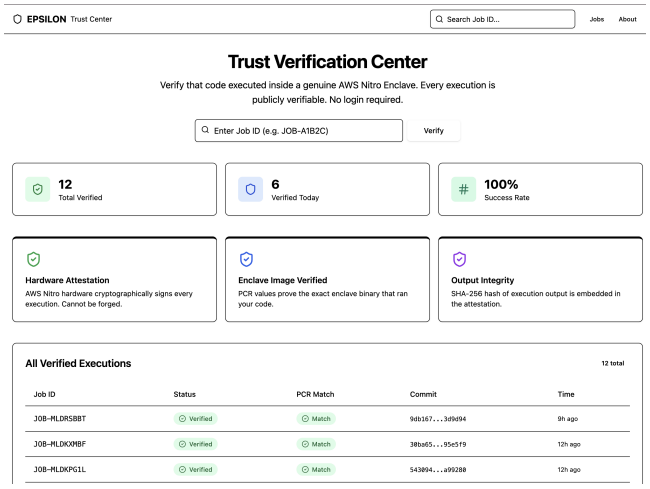

Figure 9. Verification Center homepage. Public ledger of all verified executions with aggregate statistics and per-job verification status. No login required.

F.2 Three-Panel Verification Interface

Each job opens a three-panel verification page (Figure 10). The **left sidebar** lists six verification sections—Hardware Attestation, Enclave Image, Certificate Chain, Execution Proof, Output Integrity, and Raw Attestation—each with a green checkmark indicating client-side verification status. A banner confirms: "All checks passed—verified client-side on your device." The **center panel** shows full details for the selected section. The **right panel** displays an interactive React Flow trust chain graph: AWS Root Certificate → Intermediate CAs → Enclave Certificate → COSE_Sign1 → Attestation Document → PCRs + User Data → Output Hash. Every node shows a green checkmark when verified.

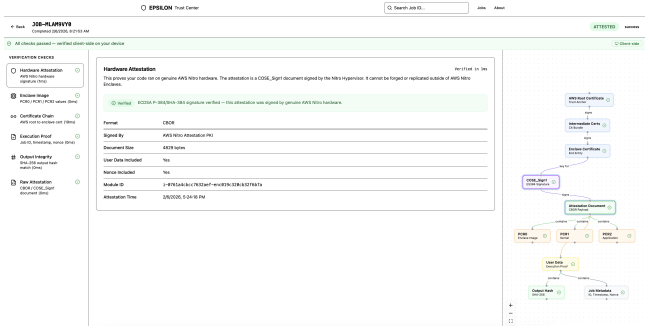

Figure 10. Three-panel verification interface. Left: six verification checks with status. Center: Hardware Attestation detail (ECDSA P-384 signature verified in 1ms). Right: interactive trust chain flow graph with color-coded nodes.

F.3 Enclave Image Verification

Figure 11 shows the Enclave Image panel displaying PCR0 (enclave image), PCR1 (kernel), and PCR2 (application) values extracted from the attestation document. All values are

compared against the published PCR registry. An educational tooltip explains: "What is PCR0? It is a SHA-384 hash of the entire Enclave Image File (EIF). If even one byte of the enclave code changes, PCR0 changes completely."

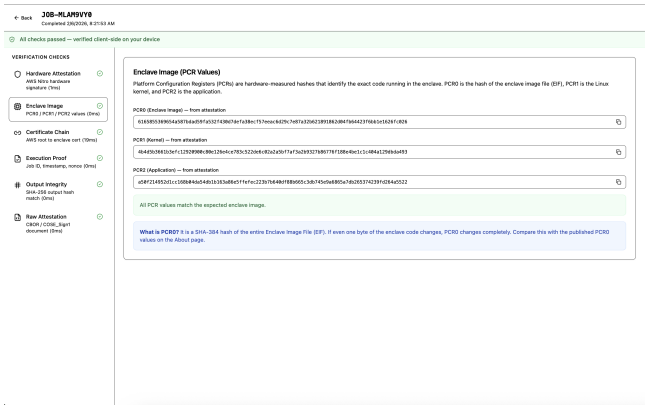

Figure 11. Enclave Image verification. PCR0/PCR1/PCR2 from the attestation document with match confirmation and educational tooltips.

F.4 Execution Proof (Per-Execution Binding)

Figure 12 shows the parsed `user_data` field from the attestation document: job ID, timestamp, module ID, output hash (SHA-256), nonce (anti-replay), commit SHA, and execution metadata (encrypted/decrypted payload sizes). This is the per-execution binding—the core Epsilon contribution—where specific code, data, and output are bound in a single TEE-signed attestation structure.

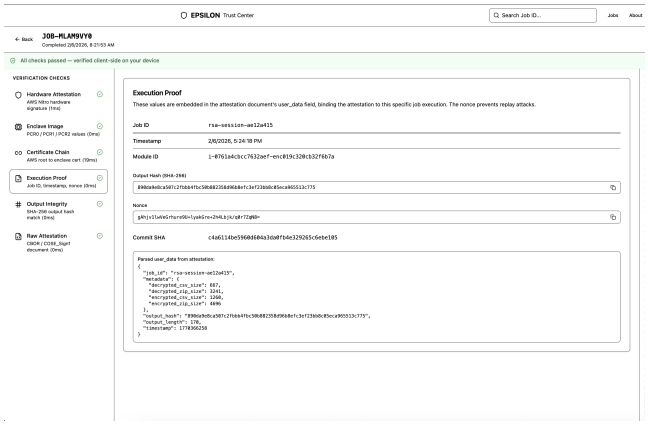

Figure 12. Execution Proof showing parsed `user_data` from the attestation. Binds job ID, output hash, nonce, commit SHA, and payload metadata in a single TEE-signed structure.

F.5 Design Rationale

The Verification Center implements two deliberate design choices. First, *all cryptographic verification runs client-side* in the visitor's browser using WebCrypto API. The platform never asks users to trust its own verification claims. The server provides raw attestation bytes; the browser independently parses CBOR, validates the certificate chain against the AWS Nitro root CA, verifies the ECDSA P-384 signature, and compares PCR values. Second, *no authentication is required.* Any party with a job ID—researcher, journal reviewer, ethics board, or auditor—can independently verify execution provenance. This supports reproducibility workflows where verification must be possible without platform accounts or API keys.

G SDK Workflow

```
# --- Setup ---
$ pip install epsilon-sdk
$ epsilon login                  # Keycloak OAuth
$ epsilon init <dataset_id>      # Generate project

# --- Analysis (main.py) ---
from generated.models import create_dataset
def main():
    dataset = create_dataset()
    ages = [r.patient.age for r in dataset]
    avg_age = sum(ages) / len(ages)
    return {"avg_age": avg_age}

# --- Test and Deploy ---
$ epsilon run                    # Local (synthetic data)
$ epsilon build                  # Package for enclave
```

Listing 2. Complete Epsilon workflow: setup, analysis, and deployment.

Acknowledgements

Claude (Anthropic) was used to assist with drafting and editing the manuscript text. All research contributions, system design, implementation, and evaluation are the sole work of the authors.